Course Name - Advanced Programming (OOP) Lab

Project Title - Library Management System

Name – Saptadip Mukherjee

Roll Number – 68

Enrollment No. - 12024052017070

Instructor Name – Dr. Soumadip Biswas

Spring 2026

Abstract

This project presents the design and implementation of a Library Management System (LMS) developed using Java without any external frameworks. The system is designed to manage books, users, and borrowing transactions efficiently using CSV files as a persistent storage mechanism.

The application follows a modular and layered architecture, separating data handling and business logic through the use of model and service layers. It incorporates core Object-Oriented Programming (OOP) principles such as encapsulation, abstraction, and composition to ensure maintainability and scalability.

Introduction

A Library Management System is essential for organizing and maintaining records of books, users, and transactions in a structured manner. This system provides functionalities such as:

- Book management (add, update, delete, search)
- User management
- Book issuing and returning
- Fine calculation for overdue returns

The system is implemented as a local Java application, making it lightweight and easy to deploy without requiring a database or web framework.

System Architecture

The system follows a layered architecture:

3.1 Model Layer

Responsible for:

- Data storage (CSV files)
- Data parsing and persistence

3.2 Service Layer

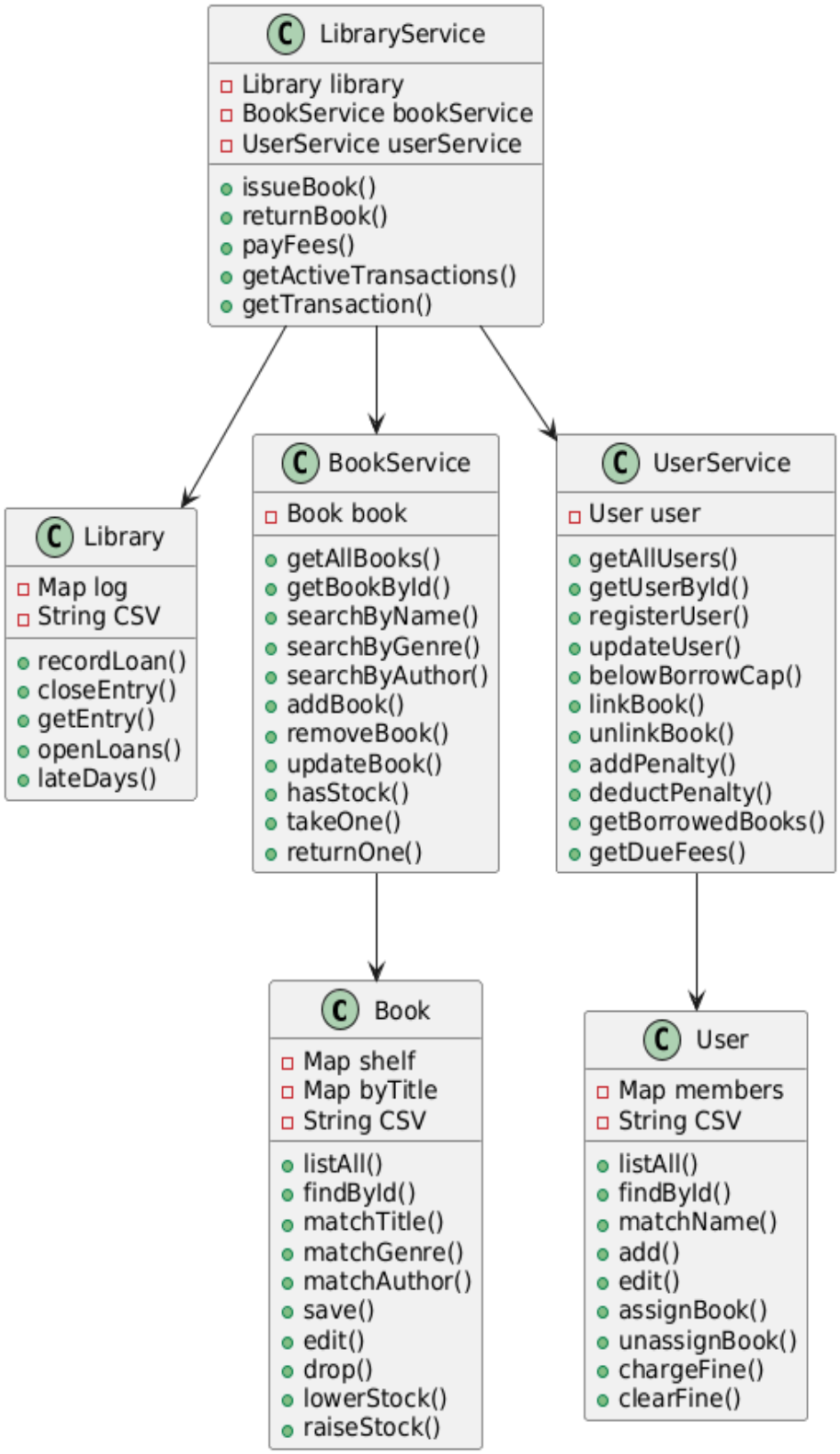
Responsible for:

- Business logic
- Validation and rules enforcement

3.3 Main Layer

- Entry point of the application
- Integrates all services

UML Class Diagram



Object-Oriented Programming Concepts

Encapsulation

Encapsulation is implemented by restricting direct access to data structures and exposing controlled interfaces.

```
private Map<Integer, Map<String, String>> shelf;

public Map<Integer, Map<String, String>> listAll() {
    return shelf;
}
```

Explanation:

- Data is declared as private
- Access is provided via public methods
- Prevents unauthorized modification

Abstraction

Abstraction is achieved by hiding implementation details inside service classes.

```
public boolean addBook(String name, String genre, String author, String year, int copies) {
    int id = book.nextAvailableId();
    return book.save(...);
}
```

Explanation:

- User interacts with simple methods
- Internal CSV handling is hidden

Polymorphism

Polymorphism is demonstrated through method variations handling different inputs.

```
searchByName(String query)
searchByGenre(String query)
searchByAuthor(String query)
```

Explanation:

- Same operation (search)
- Different implementations based on criteria

Inheritance

Inheritance is not used in this system.

Justification:

- No hierarchical relationships exist
- Composition is more suitable for this design

Design Justification

Use of CSV Files

- Lightweight and easy to manage
- No external dependencies
- Suitable for small-scale applications

Separation of Concerns

- Models handle data
- Services handle logic
- Improves maintainability

Use of Maps

- Flexible schema
- Easy parsing from CSV
- Faster lookup operations

Service-Based Architecture

- Centralized business logic
- Avoids duplication
- Enhances scalability

Testing and Edge Case Analysis

Book Module

Test Case	Expected Output
Search non-existent book	Empty list
Remove invalid ID	False
Add duplicate book	Allowed

User Module

Test Case	Expected Output
Borrow more than limit	Blocked
Invalid user ID	Null
Duplicate registration	Allowed

Transaction Module

Edge Case	Handling
Book not available	"no copies available"
User not found	"user not found"
Duplicate borrow	Prevented
Already returned	Blocked

Fine Calculation

- Late days calculated safely
- Negative values handled
- Fine = Late Days × Rate

File Handling

- Exception handling implemented

- Prevents crashes on file errors
- Ensures data consistency

Git Commit History

Commit 1: Data Files

- Added CSV files:
    - books.csv
    - users.csv
    - library.csv
- Established data storage layer

Commit 2: Models

- Implemented:
    - Book.java
    - User.java
    - Library.java
- Features:
    - File parsing
    - CRUD operations
    - In-memory storage

Commit 3: Services

- Implemented:
    - BookService.java
    - UserService.java
    - LibraryService.java
- Features:
    - Business logic

    - Validation rules
    - Borrowing system

Commit 4: Main Integration

- Integrated all components
- Enabled complete workflow
- Final system execution ready

Conclusion

The Library Management System successfully demonstrates:

- Effective use of Object-Oriented Programming
- Clean layered architecture
- Robust file-based persistence system
- Real-world features like:
    - Borrow limits
    - Fine management
    - Transaction tracking

The system is scalable and can be extended with additional features such as GUI or database integration.